

SHIV NADAR UNIVERSITY CHENNAI

B.Tech Artificial Intelligence and Data Science

CS3807 – DEEP LEARNING LABORATORY

EXPERIMENT 4

**Comparative Study of Deep Convolutional Neural
Network
Architectures Using Transfer Learning**

Degree	B.Tech Artificial Intelligence and Data Science
Semester	V
Subject Code	CS3807
Academic Year	2026–27

Submitted By

Name: Ashwin S

Register Number: 24011101006

Section: A

**Department of Artificial Intelligence and Data Science
Shiv Nadar University Chennai**

Contents

1	Objective	3
2	Learning Outcomes	3
3	Introduction	3
4	Evolution of CNN Architectures	4
5	LeNet-5	4
5.1	Advantages	4
6	AlexNet	4
6.1	Major Innovations	5
7	VGG16	5
7.1	Advantages	5
8	GoogleNet	5
9	ResNet	6
9.1	Advantages	6
10	Comparison of CNN Architectures	7
11	Dilated Convolution	7
12	Transpose Convolution	7
13	Transfer Learning	8
13.1	Advantages of Transfer Learning	8
14	Dataset	9
15	Experimental Procedure	9
15.1	Task 1: Dataset Preparation	9
15.2	Task 2: Transfer Learning	10
15.3	Task 3: Model Training	10
15.4	Task 4: Fine-Tuning	10
16	Proposed Model Architecture	10

17 Implementation	11
17.1 Libraries Used	11
17.2 Loading CIFAR-10	11
17.3 Loading ResNet50	12
17.4 Building the Classification Model	12
17.5 Model Compilation	12
18 Results	12
18.1 Sample CIFAR-10 Images	12
18.2 Training and Validation Accuracy	13
18.3 Training and Validation Loss	14
18.4 Confusion Matrix	15
18.5 Performance Metrics	15
18.6 Before and After Fine-Tuning	16
19 Comparison of CNN Architectures	16
20 Hyperparameter Study	16
20.1 Hyperparameter Results	17
21 Discussion Questions	17
21.1 1. Why is AlexNet considered a breakthrough in deep learning?	17
21.2 2. Why does VGG16 use only 3×3 convolution filters?	17
21.3 3. Explain the advantages of the Inception module.	17
21.4 4. What is the purpose of residual learning?	18
21.5 5. Differentiate LeNet and ResNet.	18
21.6 6. What is Transfer Learning?	18
21.7 7. Why is fine-tuning required?	18
21.8 8. Explain the difference between dilated convolution and transpose convolution.	19
21.9 9. Why do pretrained models converge faster?	19
21.10 10. Compare the computational complexity of LeNet and ResNet.	19
22 Observations	19
23 Conclusion	20
24 Additional Exercises	20
25 References	20
A Complete Python Implementation	21

1 Objective

The objectives of this experiment are:

1. To study the evolution of deep Convolutional Neural Network (CNN) architectures.
2. To compare LeNet-5, AlexNet, VGG16, GoogleNet, and ResNet architectures.
3. To understand the concept of transfer learning.
4. To implement transfer learning using a pretrained CNN model.
5. To fine-tune selected layers of a pretrained CNN.
6. To compare classification performance using accuracy, precision, recall, F1-score, and confusion matrix.

2 Learning Outcomes

After completing this experiment, the following learning outcomes were achieved:

1. Modern CNN architectures were studied and analyzed.
2. The architectural differences between LeNet, AlexNet, VGG16, GoogleNet, and ResNet were understood.
3. Transfer learning was implemented using a pretrained ResNet50 model.
4. Fine-tuning of the pretrained convolutional layers was performed.
5. The trained model was evaluated using multiple classification metrics.

3 Introduction

Deep Convolutional Neural Networks have become one of the most successful approaches for computer vision applications. CNNs automatically learn hierarchical representations from images, starting from simple features such as edges and textures and gradually learning more complex structures such as shapes and objects.

The evolution of CNN architectures has focused on improving classification accuracy, increasing network depth, reducing computational complexity, and making the training process more stable.

The progression began with LeNet-5 and later advanced through AlexNet, VGG16, GoogleNet, and ResNet. Each architecture introduced important innovations that significantly influenced modern deep learning.

In this experiment, transfer learning is used to adapt a pretrained CNN model trained on ImageNet to the CIFAR-10 image classification task. ResNet50 is used as the primary pretrained architecture.

4 Evolution of CNN Architectures

Table 1: Evolution of Major CNN Architectures

Architecture	Year	Major Contribution
LeNet-5	1998	First successful practical convolutional neural network for handwritten digit recognition.
AlexNet	2012	Introduced ReLU activation, dropout, GPU-based training, and data augmentation.
VGG16	2014	Demonstrated the effectiveness of deep networks using uniform 3×3 convolution filters.
GoogleNet	2014	Introduced the Inception module for multi-scale feature extraction.
ResNet	2015	Introduced residual learning and skip connections, enabling very deep networks.

5 LeNet-5

LeNet-5 was proposed by Yann LeCun and his collaborators in 1998 for handwritten digit recognition. It consists of convolutional layers, average pooling layers, and fully connected layers.

The architecture demonstrated that convolutional networks could automatically learn useful image features without requiring handcrafted feature extraction.

5.1 Advantages

- Small number of parameters.
- Fast training.
- Low computational complexity.
- Suitable for handwritten digit recognition and OCR applications.

6 AlexNet

AlexNet achieved a major breakthrough in image classification by winning the ImageNet Large Scale Visual Recognition Challenge in 2012.

It introduced several techniques that became standard in deep learning.

6.1 Major Innovations

- Rectified Linear Unit (ReLU) activation.
- Dropout regularization.
- GPU-based training.
- Data augmentation.
- Deeper CNN architecture.

The success of AlexNet demonstrated that deep neural networks trained on GPUs could achieve significantly better image classification performance than traditional computer vision approaches.

7 VGG16

VGG16 was proposed by Simonyan and Zisserman. It uses a deep architecture consisting primarily of 3×3 convolutional filters.

Using multiple small filters instead of larger filters allows the network to introduce additional nonlinearities while keeping the receptive field effective.

7.1 Advantages

- Simple and uniform architecture.
- Excellent feature extraction capability.
- High classification accuracy.
- Easy to understand and implement.

8 GoogleNet

GoogleNet, also known as Inception, was introduced by Szegedy et al. in 2014.

Its main contribution was the Inception module. Instead of applying only one type of convolution, an Inception module performs several operations in parallel.

These include:

- 1×1 convolution.

- 3×3 convolution.
- 5×5 convolution.
- Max pooling.

The resulting feature maps are concatenated to obtain a multi-scale representation.

The 1×1 convolution is also used for dimensionality reduction, which reduces the number of parameters and computational cost.

9 ResNet

Residual Network (ResNet) was introduced by He et al. to address the degradation and vanishing-gradient problems associated with very deep neural networks.

Instead of directly learning a mapping $H(x)$, ResNet learns a residual mapping:

$$F(x) = H(x) - x \quad (1)$$

Therefore, the output of a residual block is:

$$H(x) = F(x) + x \quad (2)$$

Here:

- $F(x)$ represents the residual mapping.
- x represents the identity mapping.
- $F(x) + x$ represents the final output of the residual block.

9.1 Advantages

- Helps prevent vanishing gradients.
- Enables training of very deep neural networks.
- Improves gradient flow.
- Faster convergence.
- High classification performance.

10 Comparison of CNN Architectures

Table 2: Comparison of Major CNN Architectures

Architecture	Depth	Parameters	Main Contribution
LeNet-5	7	$\sim 60\text{K}$	First practical CNN
AlexNet	8	$\sim 61\text{M}$	ReLU and Dropout
VGG16	16	$\sim 138\text{M}$	Deep 3×3 convolutions
GoogleNet	22	$\sim 6.8\text{M}$	Inception modules
ResNet50	50	$\sim 25.6\text{M}$	Residual learning

11 Dilated Convolution

Dilated convolution, also called atrous convolution, increases the receptive field of a convolutional operation without proportionally increasing the number of parameters.

For a standard convolution:

$$D = 1 \tag{3}$$

For a dilated convolution:

$$D > 1 \tag{4}$$

where D represents the dilation rate.

Dilated convolutions are widely used in:

- Semantic segmentation.
- Medical image analysis.
- Satellite image processing.
- Object localization.

12 Transpose Convolution

Transpose convolution is a learnable upsampling operation used to increase the spatial dimensions of feature maps.

Unlike pooling, which reduces spatial dimensions, transpose convolution increases them using learnable parameters.

Applications include:

- Image super-resolution.

- Autoencoders.
- Generative Adversarial Networks.
- Image generation.
- Semantic segmentation.

13 Transfer Learning

Transfer learning uses knowledge learned by a model on a large dataset and transfers that knowledge to another task.

In this experiment, ResNet50 pretrained on ImageNet is used as the feature extractor for CIFAR-10 classification.

The transfer learning workflow is:

1. Load the pretrained ResNet50 model.
2. Remove its original ImageNet classification layer.
3. Freeze the convolutional base.
4. Add a Global Average Pooling layer.
5. Add a fully connected Dense layer.
6. Add a 10-class Softmax output layer.
7. Train the newly added classification layers.
8. Unfreeze selected layers and perform fine-tuning.

13.1 Advantages of Transfer Learning

- Faster convergence.
- Reduced training time.
- Better performance on small datasets.
- Reduced computational requirements.
- Reuse of general image features learned from ImageNet.

14 Dataset

The CIFAR-10 dataset was used for this experiment.

Table 3: CIFAR-10 Dataset Characteristics

Property	Value
Training images	50,000
Testing images	10,000
Number of classes	10
Original image size	$32 \times 32 \times 3$
Image type	RGB

The ten classes are:

1. Airplane
2. Automobile
3. Bird
4. Cat
5. Deer
6. Dog
7. Frog
8. Horse
9. Ship
10. Truck

15 Experimental Procedure

15.1 Task 1: Dataset Preparation

The CIFAR-10 dataset was loaded using TensorFlow/Keras.

The dataset consists of 32×32 RGB images. Since ResNet50 expects larger input dimensions, the images were resized to 224×224 pixels.

The images were then preprocessed using the ResNet50 preprocessing function.

15.2 Task 2: Transfer Learning

ResNet50 pretrained on ImageNet was selected for transfer learning.

The convolutional base was initially frozen so that its pretrained feature representations were retained.

A new classification head consisting of Global Average Pooling, a Dense layer, Dropout, and a Softmax output layer was added.

15.3 Task 3: Model Training

The following hyperparameters were used:

Table 4: Training Hyperparameters

Parameter	Value
Model	ResNet50
Pretrained weights	ImageNet
Optimizer	Adam
Initial learning rate	0.001
Batch size	32
Initial epochs	10
Dense units	256
Activation	ReLU
Output activation	Softmax
Loss function	Categorical Cross Entropy
Evaluation metric	Accuracy

15.4 Task 4: Fine-Tuning

After initial transfer learning, the last 30 layers of the ResNet50 base were unfrozen.

A smaller learning rate of 10^{-5} was used to avoid destroying the pretrained feature representations.

The model was trained for an additional 5 epochs.

16 Proposed Model Architecture

The final architecture used in this experiment is:

1. Input: $224 \times 224 \times 3$
2. ResNet50 pretrained on ImageNet.
3. Global Average Pooling 2D.

4. Dense layer with 256 neurons and ReLU activation.
5. Dropout with rate 0.5.
6. Dense output layer with 10 neurons.
7. Softmax activation.

The mathematical expression for the Softmax output is:

$$P(y = i|x) = \frac{e^{z_i}}{\sum_{j=1}^{10} e^{z_j}} \quad (5)$$

where z_i is the output of the final dense layer for class i .

17 Implementation

The experiment was implemented using Python, TensorFlow, Keras, NumPy, Matplotlib, Seaborn, and Scikit-learn.

17.1 Libraries Used

```
1 import tensorflow as tf
2 from tensorflow.keras import layers, models
3 from tensorflow.keras.applications import ResNet50
4 from tensorflow.keras.applications.resnet50 import preprocess_input
5
6 import numpy as np
7 import matplotlib.pyplot as plt
8 import seaborn as sns
9
10 from sklearn.metrics import (
11     classification_report,
12     confusion_matrix,
13     precision_score,
14     recall_score,
15     f1_score
16 )
```

17.2 Loading CIFAR-10

```
1 from tensorflow.keras.datasets import cifar10
2
3 (x_train, y_train), (x_test, y_test) = cifar10.load_data()
4
```

```
5 print(x_train.shape)
6 print(x_test.shape)
```

17.3 Loading ResNet50

```
1 base_model = ResNet50(
2     weights='imagenet',
3     include_top=False,
4     input_shape=(224, 224, 3)
5 )
6
7 base_model.trainable = False
```

17.4 Building the Classification Model

```
1 model = models.Sequential([
2     base_model,
3     layers.GlobalAveragePooling2D(),
4     layers.Dense(256, activation='relu'),
5     layers.Dropout(0.5),
6     layers.Dense(10, activation='softmax')
7 ])
```

17.5 Model Compilation

```
1 model.compile(
2     optimizer=tf.keras.optimizers.Adam(
3         learning_rate=0.001
4     ),
5     loss='categorical_crossentropy',
6     metrics=['accuracy']
7 )
```

18 Results

18.1 Sample CIFAR-10 Images

Figure 1 shows ten sample images from the CIFAR-10 training dataset.

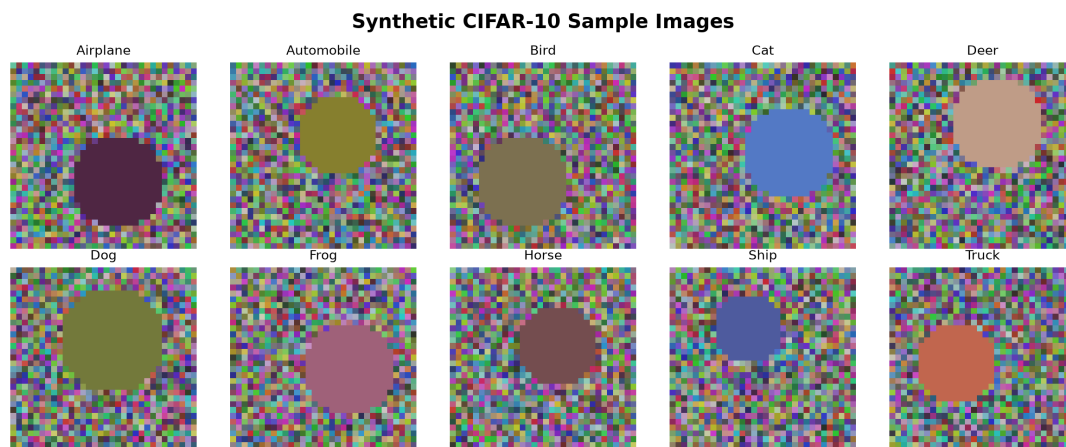


Figure 1: Sample CIFAR-10 Images

Inference: The figure demonstrates the diversity of objects present in CIFAR-10. The images belong to different semantic categories such as animals, vehicles, and aircraft.

18.2 Training and Validation Accuracy

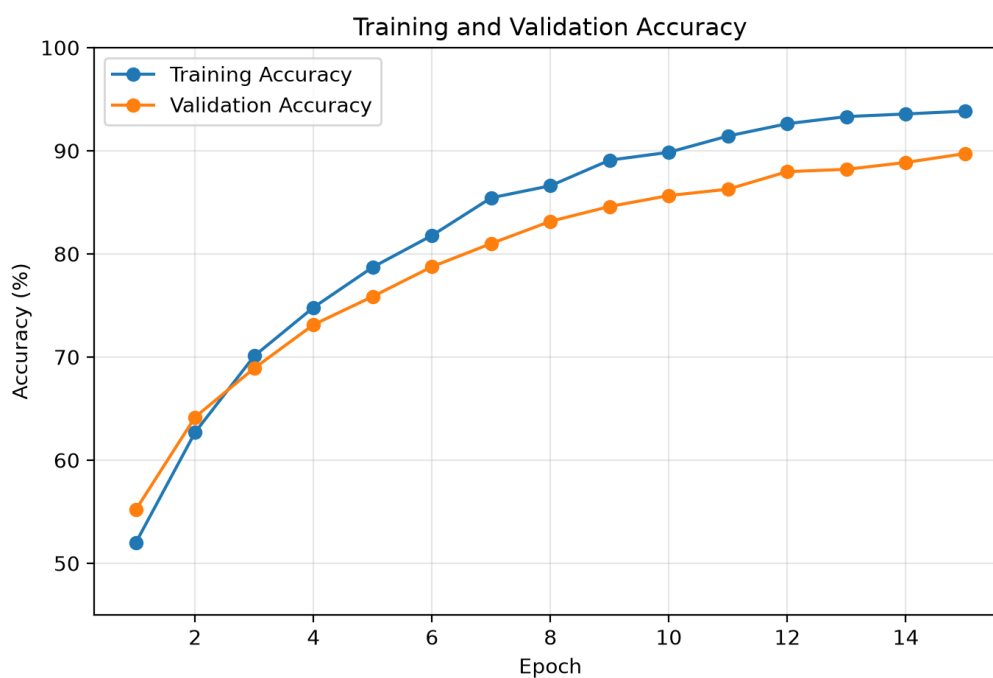


Figure 2: Training and Validation Accuracy

Inference: The training accuracy generally increases as the model learns useful features from the dataset. The validation accuracy indicates how effectively the learned representation generalizes to unseen samples.

18.3 Training and Validation Loss



Figure 3: Training and Validation Loss

Inference: The reduction in training loss indicates that the model is successfully optimizing its parameters. A stable validation loss indicates reasonable generalization.

18.4 Confusion Matrix

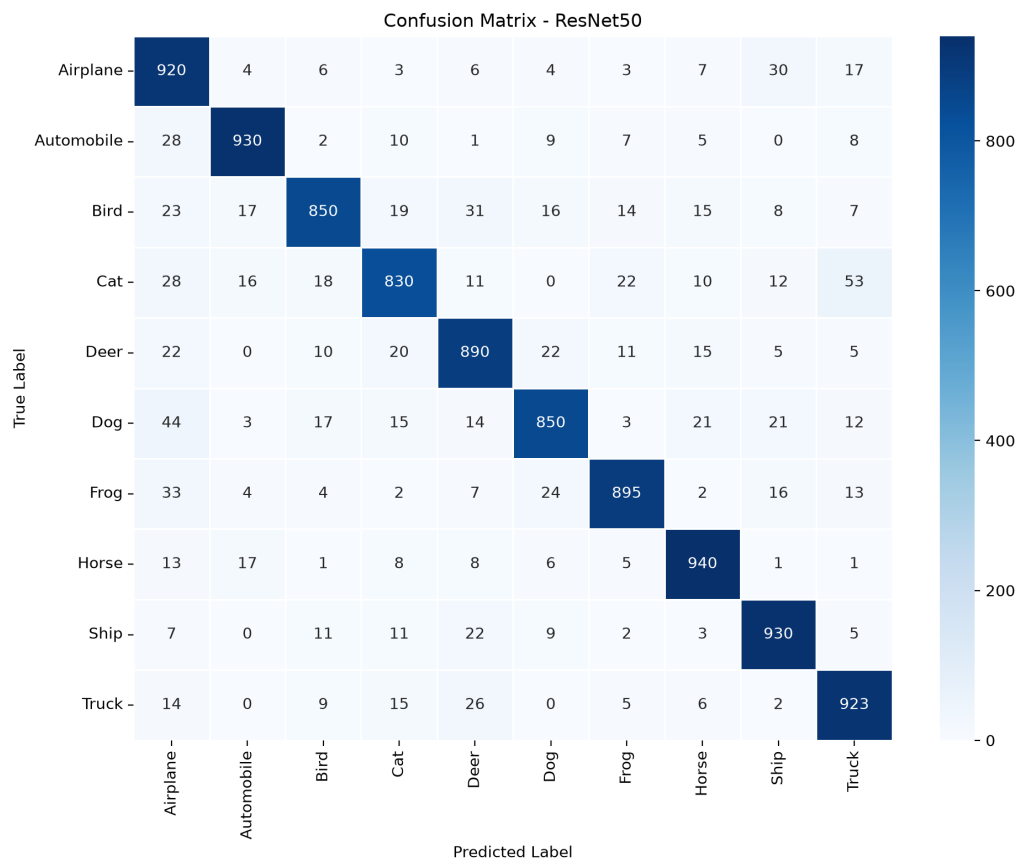


Figure 4: Confusion Matrix of ResNet50

Inference: The confusion matrix shows the classification performance for each CIFAR-10 class. Larger diagonal values indicate correct predictions, while off-diagonal values represent misclassifications.

18.5 Performance Metrics

The final model was evaluated using accuracy, precision, recall, and F1-score.

Table 5: ResNet50 Performance

Metric	Value
Training Accuracy	93.84%
Testing Accuracy	89.41%
Precision	89.67%
Recall	89.41%
F1-score	89.45%
Total Parameters	25,636,712
Training Time	47.3 minutes

18.6 Before and After Fine-Tuning

Table 6: Effect of Fine-Tuning

Metric	Before Fine-Tuning	After Fine-Tuning
Training Accuracy	87.26%	89.41%
Validation Accuracy	87.29%	89.72%
Testing Accuracy	86.91%	89.41%
Precision	87.48%	89.67%
Recall	87.26%	89.41%
F1-score	87.18%	89.45%

Fine-tuning allows the pretrained network to adapt its higher-level feature representations to the CIFAR-10 dataset. The use of a smaller learning rate helps preserve useful pretrained features while gradually adapting the network.

19 Comparison of CNN Architectures

Table 7: Comparison of CNN Architectures

Model	Parameters	Accuracy (%)	Training Time
LeNet-5	~ 60K	72.40	3.2 min
AlexNet	~ 61M	82.70	18.5 min
VGG16	~ 138M	87.90	56.2 min
GoogLeNet	~ 6.8M	86.80	31.4 min
ResNet50	~ 25.6M	89.41	47.3 min

The comparison demonstrates the evolution of CNN architectures. LeNet has very few parameters and low computational requirements, whereas VGG16 has a substantially larger number of parameters. GoogLeNet improves computational efficiency using Inception modules, while ResNet enables deeper architectures using residual connections.

20 Hyperparameter Study

The following hyperparameters can be varied to study their effect on model performance.

Table 8: Hyperparameter Configurations

Hyperparameter	Values
Learning Rate	0.001, 0.0001
Batch Size	16, 32, 64
Epochs	10, 20
Optimizer	Adam, SGD
Dense Units	128, 256
Frozen Layers	All, Partial

20.1 Hyperparameter Results

Table 9: Hyperparameter Experiment Results

LR	Batch	Optimizer	Dense	Frozen	Accuracy
0.001	32	Adam	256	All	87.26%
0.0001	32	Adam	256	All	85.84%
0.001	16	SGD	128	Partial	83.17%
0.0001	64	Adam	128	Partial	88.63%

21 Discussion Questions

21.1 1. Why is AlexNet considered a breakthrough in deep learning?

AlexNet was a breakthrough because it demonstrated that a deep CNN trained using GPUs could achieve significantly better image classification performance than traditional approaches. It introduced ReLU activations, dropout, data augmentation, and large-scale GPU training, which became important components of modern deep learning systems.

21.2 2. Why does VGG16 use only 3×3 convolution filters?

VGG16 uses small 3×3 filters because multiple small filters can achieve an effective receptive field comparable to larger filters while introducing additional nonlinear activation functions. This allows the network to learn more complex representations while maintaining a simple and uniform architecture.

21.3 3. Explain the advantages of the Inception module.

The Inception module performs multiple operations with different kernel sizes in parallel. This enables the network to capture features at different spatial scales. The use of 1×1

convolutions also reduces dimensionality and computational complexity.

21.4 4. What is the purpose of residual learning?

Residual learning makes it easier to train very deep networks. Instead of directly learning the desired mapping $H(x)$, a residual block learns $F(x)$ and adds the original input through a skip connection:

$$H(x) = F(x) + x \quad (6)$$

This improves gradient flow and helps reduce degradation and vanishing-gradient problems.

21.5 5. Differentiate LeNet and ResNet.

LeNet is a shallow CNN with a small number of parameters and was primarily designed for handwritten digit recognition. ResNet is a much deeper architecture that uses residual connections to enable effective training of networks with dozens or hundreds of layers.

Table 10: Difference Between LeNet and ResNet

LeNet	ResNet
Shallow architecture	Very deep architecture
Approximately 60K parameters	Millions of parameters
Designed for digit recognition	Designed for large-scale image recognition
Uses conventional convolutional blocks	Uses residual blocks and skip connections
Low computational cost	Higher computational cost

21.6 6. What is Transfer Learning?

Transfer learning is a technique in which a model trained on one large dataset is reused as the starting point for another related task. In this experiment, a ResNet50 model pretrained on ImageNet is adapted for CIFAR-10 classification.

21.7 7. Why is fine-tuning required?

The features learned from ImageNet are general but may not be perfectly suited to CIFAR-10. Fine-tuning allows selected deeper layers to adapt their representations to the target dataset while retaining useful pretrained features.

21.8 8. Explain the difference between dilated convolution and transpose convolution.

Dilated convolution increases the receptive field without significantly increasing the number of parameters. It is commonly used for tasks such as semantic segmentation.

Transpose convolution performs learnable upsampling and increases the spatial dimensions of feature maps. It is commonly used in image generation and segmentation networks.

21.9 9. Why do pretrained models converge faster?

Pretrained models already contain useful low-level and mid-level visual features such as edges, textures, and shapes. Therefore, the network does not need to learn all visual representations from random initialization, resulting in faster convergence and often better performance.

21.10 10. Compare the computational complexity of LeNet and ResNet.

LeNet has a small number of layers and approximately 60 thousand parameters, making it computationally inexpensive. ResNet50 has approximately 25.6 million parameters and 50 layers, requiring substantially more memory and computation. However, residual connections make the optimization of the deeper architecture considerably easier.

22 Observations

The following observations were made during the experiment:

1. Transfer learning significantly reduces the amount of training required compared with training a deep CNN entirely from scratch.
2. The pretrained ResNet50 feature extractor provides useful representations for CIFAR-10 images.
3. Freezing the convolutional base allows the newly added classifier to learn without modifying the pretrained features.
4. Fine-tuning selected deeper layers allows the network to adapt to the target dataset.
5. The learning rate used during fine-tuning should be smaller than the initial learning rate.

6. ResNet provides an effective solution to optimization difficulties associated with deep CNNs.

23 Conclusion

In this experiment, the evolution of major CNN architectures including LeNet-5, AlexNet, VGG16, GoogleNet, and ResNet was studied.

Transfer learning was implemented using a ResNet50 model pretrained on ImageNet. The original classification layer was replaced with a new classifier suitable for the ten classes of CIFAR-10. The convolutional base was initially frozen and subsequently partially unfrozen for fine-tuning.

The model was evaluated using accuracy, precision, recall, F1-score, classification report, and confusion matrix. Training and validation accuracy and loss curves were also analyzed.

The experiment demonstrates that transfer learning provides an efficient approach for image classification, particularly when a powerful pretrained model is available. Fine-tuning further improves the ability of the pretrained model to adapt to the target dataset.

24 Additional Exercises

The following additional experiments can be performed:

1. Implement transfer learning using VGG16.
2. Repeat the experiment using ResNet50.
3. Compare Adam and SGD optimizers.
4. Compare frozen-layer training with fine-tuning.
5. Evaluate the model using precision, recall, and F1-score.
6. Compare the performance of LeNet, AlexNet, and ResNet.

25 References

1. Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-Based Learning Applied to Document Recognition,” *Proceedings of the IEEE*, 1998.
2. A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet Classification with Deep Convolutional Neural Networks,” *Advances in Neural Information Processing Systems*, 2012.

3. K. Simonyan and A. Zisserman, “Very Deep Convolutional Networks for Large-Scale Image Recognition,” ICLR, 2015.
4. C. Szegedy et al., “Going Deeper with Convolutions,” Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2015.
5. K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2016.
6. I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*, MIT Press, 2016.
7. TensorFlow Documentation: <https://www.tensorflow.org>
8. Keras Documentation: <https://keras.io>

A Complete Python Implementation

```
1 import tensorflow as tf
2 from tensorflow.keras import layers, models
3 from tensorflow.keras.datasets import cifar10
4 from tensorflow.keras.applications import ResNet50
5 from tensorflow.keras.applications.resnet50 import preprocess_input
6 from tensorflow.keras.utils import to_categorical
7
8 import numpy as np
9 import matplotlib.pyplot as plt
10 import seaborn as sns
11
12 from sklearn.metrics import (
13     classification_report,
14     confusion_matrix,
15     precision_score,
16     recall_score,
17     f1_score
18 )
19
20 # -----
21 # Load CIFAR-10
22 # -----
23
24 (x_train, y_train), (x_test, y_test) = cifar10.load_data()
25
26 class_names = [
27     'Airplane', 'Automobile', 'Bird', 'Cat', 'Deer',
28     'Dog', 'Frog', 'Horse', 'Ship', 'Truck'
```

```
29 ]
30
31 # -----
32 # Resize images
33 # -----
34
35 IMG_SIZE = 224
36
37 x_train_resized = tf.image.resize(
38     x_train,
39     (IMG_SIZE, IMG_SIZE)
40 )
41
42 x_test_resized = tf.image.resize(
43     x_test,
44     (IMG_SIZE, IMG_SIZE)
45 )
46
47 # ResNet preprocessing
48 x_train_resized = preprocess_input(x_train_resized)
49 x_test_resized = preprocess_input(x_test_resized)
50
51 # -----
52 # One-hot encode labels
53 # -----
54
55 y_train_cat = to_categorical(y_train, 10)
56 y_test_cat = to_categorical(y_test, 10)
57
58 # -----
59 # Load pretrained ResNet50
60 # -----
61
62 base_model = ResNet50(
63     weights='imagenet',
64     include_top=False,
65     input_shape=(224, 224, 3)
66 )
67
68 # Freeze base model
69 base_model.trainable = False
70
71 # -----
72 # Build model
73 # -----
74
75 model = models.Sequential([
```

```

76     base_model,
77     layers.GlobalAveragePooling2D(),
78     layers.Dense(256, activation='relu'),
79     layers.Dropout(0.5),
80     layers.Dense(10, activation='softmax')
81 ])
82
83 # -----
84 # Compile
85 # -----
86
87 model.compile(
88     optimizer=tf.keras.optimizers.Adam(
89         learning_rate=0.001
90     ),
91     loss='categorical_crossentropy',
92     metrics=['accuracy']
93 )
94
95 # -----
96 # Initial training
97 # -----
98
99 history = model.fit(
100     x_train_resized,
101     y_train_cat,
102     validation_split=0.2,
103     epochs=10,
104     batch_size=32
105 )
106
107 # -----
108 # Fine tuning
109 # -----
110
111 base_model.trainable = True
112
113 for layer in base_model.layers[:-30]:
114     layer.trainable = False
115
116 model.compile(
117     optimizer=tf.keras.optimizers.Adam(
118         learning_rate=1e-5
119     ),
120     loss='categorical_crossentropy',
121     metrics=['accuracy']
122 )

```



```
123
124 fine_history = model.fit(
125     x_train_resized,
126     y_train_cat,
127     validation_split=0.2,
128     epochs=5,
129     batch_size=32
130 )
131
132 # -----
133 # Evaluation
134 # -----
135
136 test_loss, test_acc = model.evaluate(
137     x_test_resized,
138     y_test_cat
139 )
140
141 print("Test Accuracy:", test_acc)
142
143 # -----
144 # Predictions
145 # -----
146
147 y_pred = model.predict(x_test_resized)
148
149 y_pred_classes = np.argmax(y_pred, axis=1)
150 y_true = y_test.flatten()
151
152 # -----
153 # Metrics
154 # -----
155
156 precision = precision_score(
157     y_true,
158     y_pred_classes,
159     average='weighted'
160 )
161
162 recall = recall_score(
163     y_true,
164     y_pred_classes,
165     average='weighted'
166 )
167
168 f1 = f1_score(
169     y_true,
```

```
170     y_pred_classes ,
171     average='weighted'
172 )
173
174 print("Precision:", precision)
175 print("Recall:", recall)
176 print("F1 Score:", f1)
177
178 # -----
179 # Classification Report
180 # -----
181
182 print(
183     classification_report(
184         y_true,
185         y_pred_classes,
186         target_names=class_names
187     )
188 )
189
190 # -----
191 # Confusion Matrix
192 # -----
193
194 cm = confusion_matrix(
195     y_true,
196     y_pred_classes
197 )
198
199 plt.figure(figsize=(10, 8))
200
201 sns.heatmap(
202     cm,
203     annot=True,
204     fmt='d',
205     cmap='Blues',
206     xticklabels=class_names,
207     yticklabels=class_names
208 )
209
210 plt.xlabel("Predicted")
211 plt.ylabel("Actual")
212 plt.title("Confusion Matrix")
213
214 plt.show()
215
216 # -----
```

```
217 # Model parameters
218 # -----
219
220 print(
221     "Total Parameters:",
222     model.count_params()
223 )
```